

北京理工大学

本科生毕业设计（论文）外文翻译

外文原文题目: Hopter: a Safe, Robust, and Responsive
Embedded Operating System

中文翻译题目: Hopter: 一个安全、健壮且高响应能力的嵌
入式操作系统

基于协程与用户态中断的异步化 Alien 内核设计 与实现

The Design and Implementation of Asynchronous Allien based Coroutine and User-mode Interrupt

学 院:	计算机学院
专 业:	计算机科学与技术（徐特立英才班）
班 级:	07022202
学生姓名:	胡嘉晨
学 号:	1120220611
指导教师:	陆慧梅

译文：

Hopter：一个安全、健壮且高响应能力的嵌入式操作系统

摘 要

基于微控制器的嵌入式系统容易受到内存安全问题影响，但是它们必须健壮且响应迅速，因为它们常被用于无人和关键任务场景。Rust编程语言通过编译时检查为内存安全提供了一个有吸引力的解决方案，但仍未能解决栈溢出问题，反而提升了中断处理延迟。我们提出了Hopter，一个基于Rust的嵌入式操作系统（OS），它在嵌入式系统中提供内存安全、系统健壮性和高中断响应能力，同时仅需最小程度的应用适配。Hopter通过一种新颖的有限堆栈语义，将栈溢出转换为Rust panic，从而能够通过栈展开和重启从致命错误中恢复。Hopter还采用了一种被称为软锁的新机制，使得操作系统从不禁用中断。我们使用受控工作负载将Hopter与其他知名嵌入式操作系统进行比较，并记录了我们使用Hopter开发微型无人机飞行控制系统和物联网（IoT）网关系统的经验。我们认为Hopter非常适合应用于资源受限的微控制器环境，并支持实时工作负载的错误恢复。

关键词：嵌入式系统，操作系统，Rust，内存安全，健壮性，中断响应能力

目 录

摘 要	I
第 1 章 引言	1
第 2 章 背景	3
第 3 章 使用 Hopter 进行系统开发	5
3.1 开发模型	5
3.2 使用 Hopter 抽象	6
第 4 章 系统设计	8
4.1 具有有限堆栈语义的 Rust	8
4.1.1 防止栈溢出	8
4.1.2 扩展堆栈	9
4.2 从 panic 中恢复	10
4.2.1 展开堆栈	10
4.2.2 重启应用程序	11
4.3 零延迟中断处理	11
4.3.1 算法描述	11
4.3.2 Hopter 中的用例	12
第 5 章 实现	14
第 6 章 评估	17
6.1 方法	17
6.2 受控工作负载	18
6.2.1 最小系统 (Q1/Q2)	18
6.2.2 任务调度和中断处理 (Q1/Q3)	18
6.2.3 致命错误恢复 (Q4)	21
6.3 无人机飞行控制	22
6.3.1 系统大小和 CPU 负载 (Q1/Q2/Q3)	24
6.3.2 飞行中中断响应延迟 (Q3)	24

6.3.3 飞行中致命错误容错（Q4）	25
6.4 IoT 网关.....	26
第7章 相关工作	27
结 论	29
参考文献	30
附 录	34

第1章 引言

在过去的十年中，嵌入式系统的使用量日益增长，预计在未来几十年还将进一步增加更多[37-39]。由于资源限制，许多嵌入式系统采用没有内存管理单元(MMU)的微控制器，因此无法享受与虚拟内存相关的安全性。更糟糕的是，C语言作为一种内存不安全语言，一直是编程此类基于微控制器的嵌入式系统最常用的语言。因此，这些系统可能遭受与内存相关的安全漏洞，这些漏洞难以被发现，这一点从FreeRTOS[17, 18]发布十年后报告的漏洞中可见一斑。随着基于微控制器的嵌入式系统越来越多地被用于关键任务应用程序，在不要求更多资源或牺牲系统响应能力的前提下增强其内存安全性和系统健壮性势在必行。

Rust编程语言是一个有吸引力的C语言替代方案。它主要通过编译时检查来保证内存安全，仅增加少量运行时开销，并且已经在操作系统(OS)开发[8, 13, 14, 26, 50, 56]（包括嵌入式系统[41, 55, 63]）中得到采用。

然而，在基于微控制器的嵌入式系统中使用Rust也面临自身的挑战。首先，即使使用safe Rust，内存安全错误仍然存在，因为其语义假设函数调用栈具有无限大小，这对于没有虚拟内存且通常只有几十到几百KiB SRAM的基于微控制器的嵌入式系统来说尤其成问题。漏洞报告显示Rust库中存在栈溢出问题，即使是那些用于嵌入式用途的库[15, 16, 19, 22-24]。其次，Rust内置的异常处理机制，例如panic，需要一个栈展开器，而这在微控制器上通常不可用。没有栈展开器，panic会导致应用程序[41]或整个系统[11, 12]挂起或重置。最后，Rust的语言限制使得难以实现零延迟中断处理，即操作系统从不禁用中断以确保对事件的及时响应。已知的解决方案[47, 48, 57, 58]使用safe Rust是不可行的，因为它们难以通过编译时检查。§ 2详细阐述了这些挑战。

在本文中，我们试图回答以下问题：我们能否在为基于Rust的嵌入式系统带来内存安全、系统健壮性和快速中断的同时，仅要求应用程序最小程度的适配？我们通过Hopter嵌入式操作系统（§ 3）给出了肯定的答案。Hopter采用操作系统与实现语言的协同设计，以完善内存安全并实现错误恢复能力。Hopter还通过一种称为软锁的新型机制，为运行线程化任务的Rust语言编写的系统带来了零延迟中断处理能力。

Hopter通过有限堆栈语义（FS-语义）增强了Rust，实现了栈内存安全和溢出恢复能力（§ 4.1）。FS-语义将每次函数调用视为潜在的panic。在执行函数体之前，一个prologue将检查剩余的栈空间，如果不足将触发panic。如果析构函数或其被调用者导致栈溢出，Hopter将临时扩展栈以完成析构函数[35, 45, 67]，然后触发panic。这有效地利用Rust panic机制将栈溢出与其他致命错误统一起来。

然后，Hopter利用自定义的栈展开器在panic时回收资源，这允许Hopter自动重启失败的任务（§ 4.2）。在可能的情况下，Hopter执行并发重启以加快恢复速度，其中新任务启动与失败任务的栈展开并发运行。

软锁通过对每个操作系统对象的修改进行序列化来实现零延迟中断处理，避免了全局序列化队列，从而避免了unsafe Rust代码（§ 4.3）。当任务或中断处理程序获取一个未被占用的软锁时，它将获得对受保护对象的完全访问权限。否则，它将获得部分访问权限以记录其计划在对象上执行的操作，软锁将在占用结束后提交这些操作。

我们将Hopter实现并开源为针对Arm Cortex-M0和Cortex-M4架构的Rust库（§ 5）。Hopter需要一个定制的编译器来编译系统，但Rust语法保持不变且语义兼容。Hopter还支持未经修改的第三方硬件抽象层（HAL）库，允许应用程序开发人员利用不断增长的Rust生态系统。我们通过与另外两个嵌入式操作系统，即FreeRTOS和Tock（§ 6），的比较来评估Hopter。Hopter具有比FreeRTOS更低的 interrupt 处理延迟，并且更安全、更健壮。Hopter比Tock需要更少的硬件资源，并且支持缺少内存保护单元（MPU）的微控制器。我们为Crazyflie 2.1微型无人机开发了一个飞行控制系统，并为物联网（IoT）设备开发了一个网关系统。使用飞行控制系统时，Hopter的闪存使用量增加了56%，FS-语义、栈展开机制和软锁共提升了15%的CPU占用率。网关系统展示了Hopter即使在无MPU的Cortex-M0 CPU上也能抵御运行时错误的健壮性。

我们做出以下贡献：

1. Rust的有限堆栈语义，通过编译时插桩和操作系统支持，保证栈内存安全和溢出恢复能力。
2. 软锁，一种新颖的同步原语，使得在基于Rust且运行线程化任务的系统上实现零延迟中断处理成为可能。
3. Hopter嵌入式操作系统的开源实现[25]，集成了FS-语义和软锁，以提供内存安全、故障恢复和快速响应，同时仅需应用最小程度的适配。

4. 对FS-语义、栈展开机制和软锁引起的可执行程序大小和性能开销的评估，表明Hopter适合应用于资源受限的微控制器和实时工作负载。

第2章 背景

微控制器广泛应用于嵌入式系统，从简单的家用电器[34, 49, 65]到复杂的汽车控制系统[28, 31, 40]和工业自动化系统[10, 46]。其中许多系统是无人值守或关键任务的，因此健壮性和从致命错误中恢复的能力至关重要。由于成本和能源限制，微控制器上可用的硬件资源通常有限。它们通常具有数百KB到几MB的闪存用于代码和只读数据，以及几十到几百KB的SRAM用于运行时数据。因此，为微控制器设计的操作系统必须是轻量级的。

微控制器在没有虚拟内存的情况下运行。所有代码都在单一的物理地址空间中运行，闪存、SRAM和外设寄存器都被映射到其中。CPU通常直接从字节寻址的闪存中执行指令，称为就地执行（XIP），而函数调用栈和可变数据则驻留在SRAM中。微控制器上的代码通常可以不受限制地访问整个地址空间，使得系统容易受到诸如缓冲区溢出和无效指针访问之类的内存安全错误的影响，这可能导致系统不稳定或引发安全漏洞。由于应用程序任务和操作系统之间缺乏隔离，攻击者可以利用内存安全漏洞轻松获得整个系统的完全控制。

基于硬件的内存保护，例如Arm上的内存保护单元（MPU）[2]和RISC-V上的物理内存保护（PMP）[54]，在高端微控制器上可用，并且已被一些嵌入式操作系统积极利用，例如Tock[41]。不幸的是，基于硬件的保护会带来代码大小、内存使用和运行性能方面的开销。MPU/PMP允许开发人员定义最多16个具有选择性读、写或执行权限的内存区域。然而，每个内存区域必须对齐并按最接近的2的幂大小调整，从而导致内部碎片而浪费闪存或SRAM。此外，使用MPU或PMP需要通过软件中断进行系统调用切换特权模式，参数需要经过封装并由操作系统验证，增加了性能开销。相比之下，在没有此类保护机制的系统中，系统调用可以有效地实现为简单的函数调用。因此，流行的嵌入式操作系统，如FreeRTOS[1]，即使是用C这样的不安全语言编写的，也将基于硬件的保护视为可选项。

Rust编程语言是一种现代系统编程语言，在提供对硬件的直接访问的同时，保证了内存和并发安全。它为基于硬件的保护提供了一个可观的替代方案，并且没有语言带来的显著开销。近年来Rust编程语言在操作系统开发[8, 13, 14, 26, 50, 56]和嵌入式系统[41, 55, 63]中的使用率有所增加。

Rust通过其所有权模型管理资源，而不使用垃圾收集器。每个值由一个变量拥有，当变量超出作用域时，它的析构函数被执行以释放资源。与其他语言通常在将值分配给新变量时进行复制或引用不同，Rust默认将值的所有权从旧变量移动到新变量。移动后，旧变量将不可访问。Rust中的函数调用也默认将参数值的所有权移动到被调用者，被调用者负责调用这些值的析构函数。因此，调用堆栈很可能在调用析构函数时被填满，这一现象已被我们的飞行控制系统（§ 6.3）所证实。

Rust强制使用所有权模型并在编译时分析引用的生命周期以确保内存安全，除非使用unsafe关键字。重要的是，编译时检查会拒绝一些常见的代码模式。例如，只有当引用在语法范围中的生命周期超过结构的生命周期时，Rust中的结构才能包含引用。此类限制阻碍了已知的零延迟中断处理实现[47, 48, 57, 58]，这种方法需要一个全局序列化队列来存储包含对各个操作系统对象引用的待处理操作。为了解决这个问题，Hopter引入了一种称为软锁的新颖机制（§ 4.3）来实现零延迟中断处理。

Rust通过一种称为panic的语言异常机制来补充其编译时检查，以阻止那些仅在运行时才能检测到的内存错误，例如数组越界访问。调用assert!或unwrap失败的断言也会导致panic。panic会终止Rust线程的正常执行流程。在个人电脑等硬件资源丰富的系统上，发生panic之后通常会进行栈展开，遍历调用堆栈中的函数帧并调用对象的析构函数以回收资源。这使得无需重启整个系统即可从错误中恢复。然而，嵌入式Rust系统通常缺少栈展开器[11, 12, 41]，因此panic将导致系统挂起或重置。更糟糕的是，断言在嵌入式Rust代码中很常见[30, 41]。为了系统健壮性，Hopter集成了一个为微控制器优化的栈展开器（§ 4.2）。

Rust内存安全的一个漏洞是函数调用堆栈仍然可能溢出。Rust语义假设每个运行的线程具有无限的栈空间，这在微控制器上尤其不现实。检测栈溢出的常见方法，包括栈保护者（金丝雀值）[3, 32]和定期栈指针检查[4]，都是事后补救措施。在检测到时，系统内存已经被破坏。先前的基于Rust的嵌入式系统要么使用MPU/PMP来防止栈溢出[41]，接纳相关的开销，要么仅使用单一的位于SRAM下边界的向下增长的堆栈[11, 12]，这限制了调度策略。Hopter通过使用有限堆栈语义（§ 4.1）下运行

Rust代码来解决这个问题，这不仅防止了栈溢出，还将此类错误转换为Rust panic，从而允许通过栈展开和重启进行统一的恢复过程。

第 3 章 使用 Hopter 进行系统开发

在介绍Hopter的设计（§ 4）和实现（§ 5）之前，我们先描述系统开发人员如何使用它开发嵌入式系统。

3.1 开发模型

Hopter是开源的，系统开发人员可以获取这个rust库。由于Hopter支持线程化任务并且放弃了基于硬件的保护，它可以与第三方HAL库[29, 30]进行互操作，显著降低了开发工作量。开发人员必须用Rust编写应用程序代码，并使用Hopter提供的定制Rust编译器来构建将Hopter作为依赖项的系统。下载并向cargo注册定制编译器仅需三个命令。

代码 3-1 应用程序代码从`[main]`属性标记的 main 函数开始执行。其他任务可以通过指定任务构建模式动态启动。当任务入口点闭包实现 Clone trait 时，该任务可以设置为可重启任务。

```
#[main]
fn main() {
    // 为另一任务初始化资源
    // 通过Arc封装使其实现Clone trait
    let res = Arc::new(init_resource());

    // 入口点闭包也拥有Clone trait
    // 可以设置为可重启任务
    task::build()
        .set_entry(move || another_task_main(res))
        .set_priority(8)
        .set_stack_limit(1024)
        .spawn_restartable()
```

```
.unwrap();  
}
```

开发人员像往常一样使用`cargo build`编译整个系统，Rust语法保持不变。

Hopter期望应用程序代码是无害的，因为应用程序代码可能使用`unsafe Rust`，因此引入内存错误。只要应用程序使用的所有`unsafe`代码都是正确的，Hopter就能保证系统的内存安全。这与流行的FreeRTOS[1]的期望类似；我们认为这是合理的，因为基于微控制器的嵌入式系统的系统开发人员通常可以访问并修改将要编译的所有源代码。Hopter的安全模型弱于Tock[41]，在Tock中应用程序代码可能是恶意的，操作系统必须使用基于硬件的保护（及其开销）来隔离自身（见 § 2）。

3.2 使用 Hopter 抽象

Hopter为系统开发人员提供了三个重要的抽象，以实现内存安全、容错能力和快速中断响应。为了实现应用逻辑，开发人员使用可重启任务或可容错中断处理程序上下文抽象，它们保证内存安全并允许从致命错误中恢复。可容错中断处理程序对硬件事件的响应具有零延迟，并且可以通过提供的同步原语与任务协作。这里我们详细阐述这三个关键抽象：其优点以及使用方式。它们的设计和实现细节将分别在 § 4和 § 5中介绍。

可重启任务：在Hopter上运行的任务是基于线程的，并且可被抢占式调度。它们通过自动重启提高了应用程序对致命错误的恢复能力。Hopter上的应用程序使用构建器模式（如代码3-2所示）生成任务，提供入口点闭包并指定任务属性。主任务是一个例外，它从标记有提供的`#[main]`属性的函数开始执行，该函数通常初始化系统并生成其他任务。要启用自动重启，应用程序任务只需确保其入口闭包实现了`Clone trait`，并使用`spawn_restartable()`方法启动。`Clone trait`允许Hopter复制重启任务所需的环境。若闭包中捕获的所有变量都实现了`Clone trait`，则该闭包就实现了`Clone trait`。当发生栈溢出、数组越界访问或断言失败等致命错误时，可重启任务会终止，释放其资源，并自动从头开始重新执行。可重启任务不会回滚到某个显式的检查点。取而代之的是，存储在堆中或用 `static` 声明的值会在任务重启后保持不变，而函数内的局部值则会被丢弃。如果入口点闭包不可克隆，仍可使用`spawn()`方法来运行任务，但该任务在发生致命错误时只会终止并释放资源，而不会重启。

代码 3-2 使用邮箱在中断处理程序 and 任务之间进行同步。中断处理程序通过#[handler(...)]宏声明。该处理程序具有零延迟响应时间，同时仍能够调用同步原语方法。第三方 HAL crate[30] 提供外设访问。

```
// 引导后初始化为 'Some(_)'。
// 自旋锁用于满足内部可变性规则。
static TIMER: Spin<Option<CounterUs<TIM2>>> = Spin::new(None);
static MAILBOX: Mailbox = Mailbox::new();

// 由定时器IRQ周期性调用。
#[handler(TIM2)]
fn tim2_handler() {
    TIMER.lock()
        .as_mut().unwrap() // 解包'Option'
        .wait().unwrap(); // 确认IRQ
    // 恢复任务执行。
    MAILBOX.notify_allow_isr();
}

// 作为任务生成。
fn another_task_main() {
    loop {
        MAILBOX.wait(); // 阻塞并让出CPU
        // 然后做其他事情 ...
    }
}
```

可容错中断处理程序：Hopter上的中断处理程序是响应硬件中断（IRQ）的函数，它们共享一个中断堆栈，并且总是抢占任务或更低优先级的中断处理程序。它们通过在中断处理期间容忍致命错误来提高系统健壮性。应用程序使用#[handler(IRQ_NAME)]属性宏（如代码3-2所示）声明中断处理程序函数。处理程序

在发生致命错误时终止并释放资源，如果中断仍然挂起，它将重新运行。由于中断上下文中动态内存分配的限制，Hopter无法从中断栈溢出中恢复。

同步原语：Hopter以Rust结构体类型的形式为应用程序提供同步原语，以方便上下文间协调。它们允许中断处理程序和任务之间进行同步，而不影响对中断的零延迟响应时间。应用程序代码通过调用定义的方法与同步原语交互。代码2展示了使用邮箱实现定时器中断处理程序与任务之间同步的示例。硬件定时器通过第三方HAL crate[30]访问。

第 4 章 系统设计

接下来，我们介绍Hopter实现内存安全、容错能力和中断响应性的关键设计方面。Hopter采用一种基于编译器的机制来保证可重启任务和可容错中断处理程序的内存安全。值得注意的是，对于栈内存安全，Hopter在编译时插桩代码（§ 4.1）的辅助下，使用有限堆栈语义（FS-语义）执行Rust代码。FS-语义将运行时内存错误和其他致命错误统一为Rust panic，允许Hopter应用基于栈展开和重启的通用恢复机制（§ 4.2）。此外，Hopter克服了与Rust语言相关的表达能力挑战，通过一种称为软锁的新颖机制实现零延迟中断处理（§ 4.3），以支持同步原语的实现。应用逻辑对编译时插桩和操作系统内部实现保持不可知，允许使用未经修改的现有Rust库。

4.1 具有有限堆栈语义的 Rust

Hopter使用有限堆栈语义（FS-语义）执行Rust代码，以确保栈内存安全并允许系统从栈溢出中恢复。FS-语义将每个Rust执行线程与一个隐式的堆栈大小相关联。Hopter通过panic（§ 4.1.1）来防止任何在没有足够空闲堆栈空间的情况下进行的函数调用。在析构函数执行期间发生溢出的特殊情况下，Hopter会临时扩展堆栈以完成析构函数，并在之后panic（§ 4.1.2）。Hopter采取两个步骤来实现FS-语义：

4.1.1 防止栈溢出

Hopter通过在执行函数体之前检查空闲堆栈空间来检测即将发生的栈溢出。这是通过编译器在分配堆栈帧的函数体之前插入的一段序言实现的。该序言计算空闲堆栈大小，即栈指针指示的当前栈顶与存储在任务局部变量BOUNDARY中的堆栈区域

边界之间的差值。如果空闲空间不足，则陷入操作系统内核进行进一步诊断。函数序言的执行在Arm上最多需要8字节的保留堆栈空间。

Hopter在发生栈溢出的任务触发panic。该panic启动栈展开，从失败的任务中回收资源，以便在不泄露资源或死锁的情况下重启它。在通常情况下，Hopter在序言检测到即将发生的溢出时立即触发panic。陷入后，Hopter将任务的程序计数器设置为栈展开器入口以开始展开。

在析构函数执行期间发生栈溢出的罕见情况下，Hopter会扩展堆栈以在panic之前完成析构逻辑的运行。这是因为栈展开不能在析构函数内部开始，否则将跳过一些负责释放资源的代码。更准确地说，如果导致栈溢出的函数满足以下两个条件之一，Hopter将扩展堆栈并延迟panic：（1）是一个析构函数（2）直接被或间接被一个析构函数调用。

这里需要两种独立的机制来检查这些标准。第一个标准由函数序言解决，它向Hopter传递触发溢出的函数是否是析构函数。

检查第二个标准需要对析构函数进行插桩。每当一个析构函数开始执行时，它将任务局部变量IN_DROP设置为true。析构函数在完成其逻辑后将该变量重置为false。在嵌套结构类型导致嵌套析构函数调用的场景中，只有最外层的析构函数在返回前清除该标志。Hopter可以通过检查IN_DROP标志的值来确定是否满足第二个标准。需要注意的是，函数序言仍然应用于析构函数插桩之上。

如果满足任一条件时发生溢出，Hopter会推迟panic。它将任务局部变量OVERFLOWED设置为true并扩展堆栈，而不是立即panic。在完成析构逻辑后，最外层的析构函数将检查OVERFLOWED标志，并在必要时触发被延迟的panic。

由于任何函数在FS-语义下都可能溢出堆栈并启动栈展开，这与一些编译器优化相冲突。LLVM编译器后端会推断函数的nounwind属性，并基于此简化生成的代码。如果函数满足以下两个条件[42, 43]，LLVM会将其标记为nounwind：（1）函数体不包含具有副作用的指令。（2）函数仅调用nounwind函数或者是叶子函数。然后，如果调用了—个nounwind函数，LLVM通过省略着陆板和展开表项来简化代码。然而，如果这样的函数调用导致栈溢出，栈展开将失败，导致挂起或系统重置。因此，Hopter为了正确性禁用了这些编译器优化。

4.1.2 扩展堆栈

Hopter利用分段堆栈[35, 45, 67]在没有虚拟内存的系统上扩展堆栈。分段堆栈是由称为堆栈块的非连续内存块组成的链表。如果剩余堆栈空间不足，它在函数调用时动态分配一个新的堆栈块，并在函数返回时释放它。

函数序言 (§ 4.1.1) 通过向Hopter提供请求的栈帧大小和传递参数大小来进行堆栈扩展。这两个数字是编译器在编译期间已知的常量，并作为字面量嵌入指令序列中，以避免使用额外的寄存器。Hopter随后通过跟踪陷入前的程序计数器来获取这些值，然后分配一个大小不小于这两个数字之和的堆栈块。如果存在传递参数，Hopter还会将它们复制到新分配的堆栈块中，以便它们与被调用者的栈帧相邻。最后，Hopter将任务的堆栈更改为新的堆栈块，并恢复任务，以便后者继续执行函数体。

我们注意到，堆栈扩展机制需要基于软件的溢出检测。这是因为堆栈的更改必须在执行函数体之前发生。相比之下，基于硬件的保护通常是在函数体开始执行并访问无效内存地址之后才检测到溢出。在函数执行中途将执行转移到新的堆栈块非常困难，因为复制栈帧会破坏内部引用。

4.2 从 panic 中恢复

Hopter支持从任务和中断处理程序中的Rust panic中恢复。panic可能由于数组越界访问、失败的断言或FS-语义下的栈溢出而被触发。

当任务或中断处理程序panic时，Hopter终止其执行并回收已分配的资源。panic被隔离在触发它的上下文中，不会阻止系统继续执行。如果任务的入口点闭包实现了Clone trait，Hopter可以在panic时自动重启它。因此，Hopter上的中断处理程序是可容错的，任务是可重启的。

4.2.1 展开堆栈

Hopter集成了一个栈展开器，用于在panic时回收资源，从而实现任务或中断处理程序的优雅终止，并允许后续恢复。展开过程在发生panic的任务或中断处理程序的上下文中运行，在此期间调度器可以继续执行上下文切换，更高优先级的中断可以在其上嵌套。从逻辑上讲，展开器在panic时强制活动函数立即返回，从调用堆栈的顶部开始，直到任务或中断处理程序的入口函数返回。在此过程中，局部对象被析构。从机制上讲，展开器引用编译器生成的展开表来恢复寄存器并调用称为着陆板的小段代码。着陆板有两种类型：调用析构函数的清理板和终止展开过程的捕获板。

Hopter的栈展开器与现有的展开器[36, 53, 64] 在两个方面不同，以支持分段堆栈。首先，Hopter的展开器避免进行永不返回的分歧函数调用。由于展开器可能被调用来清理经历栈溢出的任务，它必须扩展堆栈以执行自身的逻辑。如果展开器进行分歧调用，所使用的堆栈块将不会被释放，从而导致内存泄漏。相比之下，先前的实现通常对着陆板进行分歧调用。其次，Hopter的展开器识别堆栈块边界，并在展开过程中释放堆栈块以防止内存泄漏。这很重要，因为任务的堆栈可能包含多个堆栈块，例如，当一个溢出的析构函数在扩展的堆栈块上运行引发延迟panic时。

4.2.2 重启应用程序

通过展开器回收资源后，Hopter可以通过从其入口点闭包重启来恢复发生panic的任务。为了启用重启，Hopter要求任务的入口点闭包实现Clone trait，以便安全地复制闭包所捕获的变量。对于中断处理程序，Hopter在捕获panic后执行异常返回，而不是重新执行处理程序。如果中断请求仍然挂起，中断处理程序将自动再次被调用。

为了加快从任务panic中的恢复，Hopter实现了[44]提出的并发任务重启优化。这种技术允许立即执行panic任务的新实例，与先前实例的展开过程并发运行，从而确保最小的无响应时间。Hopter将panic实例的任务优先级降至最低，允许展开过程利用空闲的CPU时间。Rust的所有权模型消除了重启实例与正在展开的实例之间的竞争条件。应用程序程序员可以选择退出并发重启，以防止重启的任务观察到逻辑上不一致的数据。在这种情况下，Hopter执行上下文重用优化，将已展开任务的任务结构重用于重启的实例。

4.3 零延迟中断处理

Hopter支持零延迟中断处理，在接收到中断时立即调用处理函数。这是可能的，因为Hopter从不禁用中断。为了防止中断处理程序与被抢占的上下文之间的竞争条件，Hopter采用了一种新颖的机制，称为软锁，它适合Rust的编译时检查。这些锁通过序列化来自并发上下文的修改来保护操作系统数据结构，允许Hopter中的中断处理程序与操作系统对象交互，例如，使用同步原语通知任务。

4.3.1 算法描述

软锁消除了因中断处理程序的并发访问而导致的竞态条件。获取软锁会产生对受保护数据结构的完全访问或部分访问。如果软锁尚未被获取，代码获得完全访问权限；否则获得部分访问权限。完全访问允许对所有数据结构字段进行修改，而部分访问仅允许修改部分字段。

当一个被抢占的上下文持有完全访问权限时，中断处理程序获得部分访问权限来记录其计划执行的操作。这些操作被推迟，直到处理程序返回且具有完全访问权限的代码完成其自身操作。为了进一步防止因并发任务执行而产生的竞态条件，调度器总是在获取软锁之前被挂起，因此在任务上下文中运行的代码总是获得完全访问权限。由于中断处理程序总是抢占任务并具有严格的优先级，具有完全访问权限的代码在处理完具有部分访问权限的处理程序后恢复执行。因此，延迟操作可以在具有完全访问权限的代码完成其逻辑后立即无冲突地运行。

软锁封装了受保护的数据结构，并维护两个原子布尔标志来跟踪争用状态以及是否有任何延迟操作待处理。代码4-1展示了获取和释放软锁的伪代码。`locked`标志指示数据结构是否处于争用状态，`pend`标志指示是否有延迟操作待处理。受保护的数据结构必须实现`AllowDeferredOp trait`，以指定哪些字段可以通过完全或部分访问器访问，以及用于运行延迟操作的代码。在释放软锁时，会检查待处理的延迟操作，并在必要时使用作用域完全访问器执行它们，这使得它适合编译时检查。在实际实现中，软锁通过访问守卫的析构函数自动释放，而非手动释放。

4.3.2 Hopter 中的用例

软锁保护Hopter中的四种数据结构。表5-1列出了这些数据结构，以及完全或部分访问下的操作和延迟操作。等待队列为实现互斥锁、条件变量、信号量和通道等同步原语提供了基础。就绪队列为调度器维护就绪任务。邮箱是一种轻量级同步原语，允许任务等待通知，可以选择设置超时时间。休眠队列存储等待虚拟定时器到时的休眠任务。中断处理程序可以通过从等待队列或邮箱中移除任务、将其添加到就绪队列，并可选地从休眠队列中删除它来通知任务。

代码 4-1 软锁获取和释放操作的伪代码。受保护的数据结构必须实现 `AllowDeferredOp trait`，以指定哪些字段可以通过 `Full` 或 `Partial` 访问器访问，以及在运行 `deferred_op()` 时执行什么代码。软锁在无争用时返回 `Full` 访问器，否则返回 `Partial`。延迟操作在释放 `Full` 访问器时执行。循环避免了错过在第一次检查 `pend` 标志之后到来的延迟操作。

```
struct SoftLock<T> where T: AllowDeferredOp{
    locked: AtomicBool, pend: AtomicBool, ...
}

fn acquire(sl: &SoftLock<T>) -> Access{
    let prev_locked = sl.locked.swap(true);
    if prev_locked { return Access::Partial; }
    else { return Access::Full; }
}

fn release(sl: &SoftLock<T>, acc: Access){
    match acc{
        Access::Partial => { sl.pend = true; }
        Access::Full => {
            loop{
                let prev_pend = sl.pend.swap(false);
                if prev_pend { acc.deferred_op(); }
                sl.locked = false;
                if !sl.pend { break; }
                else { sl.locked = true; }
            }
        }
    }
}
```

当一个受保护的数据结构处于争用状态时，处理程序使用部分访问来记录其计划执行的操作，这些操作被延迟到稍后执行。例如，考虑一个在邮箱上阻塞以等待硬件事件的任务。任务首先获取邮箱的完全访问权限以进入占位符H。如果中断发生在修改H期间，处理程序将获得对邮箱的部分访问权限，该权限仅允许其递增通知计数器x。处理程序返回后，任务恢复其操作，但在释放完全访问权限时会注意到递增

的计数器。然后任务递减计数器并取消其阻塞。在另一种情况下，如果中断发生在任务完成其操作之后，任务将已经释放了完全访问权限并阻塞了自身。处理程序将获取邮箱的完全访问权限并在H中解除对任务的阻塞。

第5章 实现

我们已经为Arm Cortex-M4和Cortex-M0架构实现了Hopter，代码量约为15,000行。要在任一架构的微控制器系统上使用Hopter，开发人员只需定义一个中断向量数组，因为Hopter利用现有的HAL库进行外设访问。实现由两部分组成：编译器修改和Rust库代码。此外，我们以分层方式实现了Hopter的特性，以便开发人员可以在其收益和开销之间进行权衡。

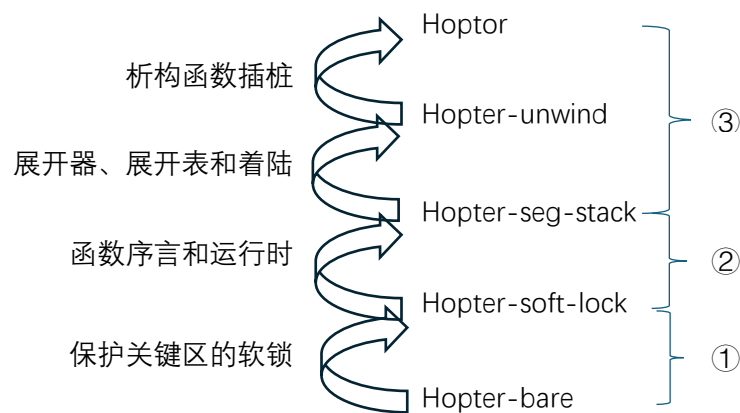
表 5-1 由软锁保护的数据结构。如果软锁没有处于争用状态，代码获得对数据结构的完全访问权限以执行所列操作之一。在任务上下文中运行的代码总是获得完全访问权限。如果在被抢占的上下文中正在访问数据结构，中断处理程序获得部分访问权限，在这种情况下，处理程序只能访问带下划线的字段以记录其计划操作。延迟操作在代码释放完全访问权限之前立即运行。

数据结构	字段	完全访问	部分访问	延迟操作
等待队列	链表 L 计数器 <u>x</u>	1. 将任务添加到 L 2. 从 L 中移除最高优先级任务	递增 <u>x</u>	从 L 中移除 <u>x</u> 个最高优先级任务并清除 <u>x</u>
就绪队列	链表 L 无锁缓冲区 <u>!</u>	1. 将任务添加到 L 2. 从 L 中移除最高优先级任务	将任务插入 <u>!</u>	将 <u>!</u> 中的任务添加到 L 并清除 <u>!</u>
邮箱	任务占位符 H 计数器 <u>x</u>	1. 设置 H 以持有任务或递减 <u>x</u> 2. 从 H 中取出任务或递增 <u>x</u>	递增 <u>x</u>	如 H 不为空，则清除 H 并递减 <u>x</u>
休眠队列	链表 L 无锁缓冲区 <u>D</u>	1. 将任务添加到 L 2. 从 L 中移除到时任务	将任务插入 <u>D</u>	从 L 中移除到时任务和 <u>D</u> 中的任务，然后清除 <u>D</u>

大部分实现是架构无关的。只有两个很小的部分不是：为生成函数序言而对编译器的修改，以及库中的内联汇编。尽管Cortex-M0支持的所有指令在M4上也是合法的，但我们使用了一些仅M4的指令来减少代码大小并提高M4上的性能。由于M0缺少原子指令，如ldex和strex，我们使用全局中断屏蔽为Cortex-M0实现原子更新操作。

编译器修改：我们修改了rustc编译器前端和LLVM后端以支持FS-语义，我们开源了修改补丁。为了生成函数序言（§ 4.1.1），我们在rustc的中间表示（IR）生成阶段向所有函数添加了split-stack属性。该属性触发LLVM帧降低阶段中的序言调整，我们进一步修改了该步骤以生成我们想要的序言。为了对析构函数进行插桩（§ 4.1.1），我们修改了rustc的中级IR（MIR）阶段的析构函数展开步骤。根据统计，我们向前端添加了260行代码，向后端添加了270行代码。我们还分别从前端和后端移除了30行和110行代码，以防止基于nounwind属性的优化（§ 4.1.1）。最后，我们修改了Rust核心库中的50行代码，以去除PanicInfo中未使用的调试信息字段，这可以将编译后的二进制文件大小减少几KB。

Rust库：我们将Hopter实现为一个Rust库，包含大约12,200行Rust代码，其中只有不到1,000行是unsafe Rust代码。Hopter仅在特定功能无法用safe Rust表达时才诉诸于unsafe Rust代码，例如堆栈扩展和动态内存管理。unsafe Rust代码还包括用于直接寄存器访问的内联汇编、引导、中断处理程序入口点以及原始内存操作（如memset）。我们还实现了两个辅助库：一个为Hopter提供#[main]和#[handler]属性宏，另一个允许应用程序配置操作系统参数。它们分别包含大约1,000行和40行的safe Rust代码。此外，Hopter依赖于开源的Rust库来实现自旋锁[66]、侵入式数据结构[21]、无锁数据结构[20]以及访问CPU核心外设[33]。尽管这些库在其提供的安全抽象内部包含unsafe Rust代码，但它们已被Rust社区广泛使用和审查，每个库的下载量至少达数百万次。



①中断响应能力 ②内存安全
③系统健壮性 ②+③ Rust 增强

图 5-1 Hopter 特性的分解。底部的“基础”版本不包含任何组件，而顶部版本包含所有组件。
层次结构中的每个版本比其下方的版本多包含一个特性。

特性层次结构：我们以分层方式使Hopter的每个特性成为可选的，如图5-1所示。该层次结构从"bare"版本开始，该版本不包含任何特性。向上，"soft-lock"版本通过用软锁替换临界区中的中断屏蔽来实现零延迟中断处理。"seg-stack"版本通过为每个编译函数生成序言并结合堆栈扩展运行时，确保栈内存安全并允许堆栈扩展。"unwind"版本通过结合定制的栈展开器，支持在Rust panic时回收资源。编译器还生成展开表和着陆板以协助展开器。最后，完整的Hopter版本通过对析构函数进行插桩并禁用基于nounwind属性的编译器优化，使得系统能够抵御栈溢出。

Cortex-M0的原子操作：由于M0不支持原子指令，特别是ldex和strex，我们通过在局部范围内禁用中断来实现Cortex-M0上的原子更新方法，因为Hopter需要这些方法来实现软锁和使用标准引用计数类型Arc。这些方法包含比较并交换（CAS）和获取并更新（例如，fetch_add），它们被定义为原子类型（如AtomicBool和AtomicPtr）和原子整数类型（如AtomicU32）上的方法。清单4展示了CAS逻辑的伪代码。获取并更新操作的代码类似。这向Rust core库添加了大约900行代码。

代码 5-1 为 Cortex-M0 实现的原子比较并交换操作的伪代码。该操作将原子整数的当前值与期望值进行比较，如果匹配，则存储新值。禁用中断确保了操作的原子性。

```
fn compare_and_exchange(
    x: &mut AtomicU32, expect: u32, new: u32
) -> (bool, u32) {
    // 禁用中断
    let irq_disabled = is_irq_disabled();
    disable_irq();

    // CAS 逻辑
    let cur = x.val; let succ = false;
    if expect == cur { x.val = new; succ = true; }
```

```
// 启用中断
if !irq_disabled { enable_irq(); }

return (succ, cur);
}
```

通过禁用中断实现原子更新操作对Hopter的中断处理延迟影响很小，这已被我们的测量结果（图6-1）所证实。更重要的是，中断仅在执行原子逻辑的短时间内被禁用，这段时间少于10条指令。因此，Hopter的中断处理延迟仍然是常数量级的。

第6章 评估

Hopter旨在提供系统内存安全、健壮性和高响应能力，同时仅需应用最小程度的适配。由于内存安全是通过架构实现的，本节量化健壮性和响应能力，以及Hopter设计带来的开销。具体来说，我们试图回答以下问题：

（Q1 开销）：为实现其目标，Hopter引入了多少开销？

（Q2 大小）：Hopter是否足够轻量，以支持具有小闪存和SRAM的微控制器？

（Q3 响应能力）：Hopter能否支持时间敏感的任务和中断处理？

（Q4 健壮性）：Hopter能否比其他操作系统从更多致命错误中恢复，并且恢复的过程对于关键任务应用来说足够快？

6.1 方法

我们采用三种正交的方法来回答这些问题。

首先，为了量化Hopter每个特性的影响，我们遵循图1中的层次结构，并测量每个列出变体的统计数据。

其次，我们将Hopter与两个知名的嵌入式操作系统进行比较：最广泛使用的嵌入式操作系统FreeRTOS[1]，以及最先进的基于Rust的嵌入式操作系统Tock[41]。我们为所有三个操作系统开发了一套具有受控工作负载的微基准测试。FreeRTOS不采用任何机制来提供内存安全或抵御应用程序错误的健壮性，其设计旨在支持高频工作负载和轻量级。Tock假设应用程序代码可能是恶意的，并采用基于硬件的保护。

最后，为了评估Hopter在真实应用下的性能和开销，并展示其在关键任务场景下从错误中恢复的能力，我们使用Hopter开发了两个嵌入式系统：为Crazyflie[6]微型无人机开发的飞行控制系统和物联网（IoT）网关。

6.2 受控工作负载

我们使用两个评估板进行微基准测试：配备Arm Cortex-M4 CPU、256 KiB SRAM和1 MiB闪存的STM32F412G-Discovery板，以及配备M0 CPU、16 KiB SRAM和128 KiB闪存的STM32F072B-Discovery板。我们将M4 CPU配置为96 MHz工作频率，M0为48 MHz工作频率，并启用M4上的浮点单元。我们将编译器设置为优化运行速度（-O3）。在我们的实验中，我们首先构建一个LED闪烁应用程序，以显示使用操作系统进行开发所需的最小资源。接下来，我们通过测量任务上下文切换的延迟和对中断的响应延迟来量化系统响应能力。最后，我们通过一个串行服务器任务证明Hopter能够从比其他操作系统更复杂的场景中恢复。

6.2.1 最小系统（Q1/Q2）

为了测量使用三种操作系统开发的系统所需的最小硬件资源，我们构建了一个闪烁LED应用程序，这是嵌入式世界的"Hello World"程序。我们扮演应用程序开发人员的角色，通过公共API使用操作系统和硬件抽象层（HAL）库，而不修改其内部实现。我们分别使用stm32-rs[30]和STM32Cube[61, 62]作为Hopter和FreeRTOS的HAL库。Tock有自己的HAL库。

表6-1中的"最小系统"一列列出了闪存和SRAM的大小。Hopter的闪存开销主要来自栈展开器逻辑、展开表、着陆板和析构函数插桩。函数序言和堆栈扩展运行时带来的开销较小。对于一个更复杂的应用程序（如飞行控制系统（§ 6.3）），闪存开销可以被平摊，我们提供了其56%闪存开销的详细分解。

运行没有健壮特性的Hopter（即"bare"、"soft-lock"或"seg-stack"版本）所需的最小硬件资源与FreeRTOS类似。对栈展开和析构函数插桩的支持产生了明显的开销，特别是在闪存使用方面，但仍然保持在几十KiB的范围内。另一方面，如表6-1所示，使用Tock开发的系统需要非常大的闪存和SRAM，因为Tock是与应用程序分开编译的。编译器工具链在编译时无法移除未使用的操作系统代码，导致体积膨胀。

6.2.2 任务调度和中断处理（Q1/Q3）

为了比较系统响应能力，我们测量以下性能指标：两个任务之间的上下文切换延迟以及处理程序或任务响应中断的延迟。

表 6-1 将 Hopter 与 FreeRTOS 和 Tock 进行比较，展示 Hopter 组件的开销，并展示通过禁用中断实现原子操作的开销。Hopter 需要比 FreeRTOS 更多的闪存，主要是因为它包含了额外的组件来提高健壮性。Hopter 比将操作系统代码分开编译的 Tock 需要显著更少的闪存和内存。得益于软锁，Hopter 具有最低且最稳定的中断响应延迟。Hopter 的上下文切换延迟和任务通知延迟接近 FreeRTOS，比 Tock 低一个数量级。Hopter 的大部分闪存和性能开销来自提供健壮性的组件，而响应能力和安全性组件仅引入很小的开销。Hopter 在 Cortex-M0 上像在 M4 上一样提供所有三个特性，但通过禁用中断实现原子操作，使上下文切换和中断处理的延迟增加了 20%。

		最小系统		延迟 (μs)			特性		
		闪存 (KiB)	†SRAM (KiB)	*任务	**中断	**中断-任务	高响应能力	安全	健壮
操作系统	FreeRTOS	13.70	3.76	8.0	2.20(0.39)	12.76(0.73)	✓	×	×
	Tock	‡147.5+6.60	‡66.53+0.75	264.7	151.54(26.03)	365.14(26.71)	×	✓	✓
	Hopter	27.68	1.00	16.0	1.36(0.01)	31.88(2.44)	✓	✓	✓
组件	Hopter-unwind	22.71	1.00	12.9	1.32(0.03)	26.44(1.96)	✓	✓	×
	Hopter-seg-stack	12.60	0.84	13.0	1.20(0.01)	27.24(1.90)	✓	✓	×
	Hopter-soft-lock	10.05	0.75	12.6	1.16(0.02)	25.48(1.84)	✓	×	×
	Hopter-bare	9.12	0.49	15.5	5.26(1.21)	28.30(1.83)	×	×	×
架构	FreeRTOS (M0)	12.36	3.75	19.2	4.88(1.15)	29.50(4.43)	✓	×	×
	FreeRTOS (M0 on M4)	13.32	3.75	7.3	2.12(0.43)	12.22(1.66)	✓	×	×
	Hopter (M0)	25.60	0.92	48.7	3.18(0.05)	76.20(7.03)	✓	✓	✓
	Hopter (M0 on M4)	25.67	0.92	19.9	1.64(0.02)	38.64(2.83)	✓	✓	✓

†：不包括函数调用堆栈，其大小可配置。‡：操作系统加上应用程序大小。

*：10,000次试验的平均值。**：10,000次试验的最大值和标准差。

M0：在 STM32F072B-Discovery (Cortex-M0) 上测量。其他在 STM32F412G-Discovery (Cortex-M4) 上测量。M0 on M4：代码仅使用 M0 指令编译，但在 STM32F412G-Discovery (M4) 上运行。

上下文切换延迟反映了任务间协作的开销。Hopter的延迟与FreeRTOS处于同一数量级，使其能够在高达1000Hz的频率下支持多个协作任务。而Tock显著更慢，因为每次任务上下文的切换需要在操作系统和应用程序之间进行四次上下文切换，导致Tock不适合用于性能要求高的工作负载。我们计算上下文切换延迟的方法是让两个任务使用最有效的可用同步原语（即Hopter上的邮箱、FreeRTOS上的任务通知、Tock上的进程间通信（IPC））相互进行上下文切换。表6-1中的"任务"一行通过使用微控制器上的精确到1 μ s的硬件计时器测量10,000次上下文切换，得到了平均延迟。Hopter的延迟高于FreeRTOS，主要是由于在实现任务列表和记录当前运行任务时使用了safe Rust。在FreeRTOS中找到的任务列表的组织 and 操作优化与Rust的所有权模型不兼容（见§2）。然而，对于以1,000 Hz运行的任务，上下文切换中8 μ s的开销导致CPU负载增加不到1%。

我们还测量了中断响应延迟，它决定了系统是否会错过短暂事件。被测板注册了一个中断处理程序，由通用输入/输出（GPIO）引脚的上升沿触发。处理程序只是将另一个GPIO引脚设置为高电平来响应，可以直接设置，也可以通过通知高优先级任务来设置。我们编程另一块板来触发上升沿，并以0.02 μ s的精度测量响应延迟。

表6-1中的"中断"一行展示了在处理程序直接响应中断的10,000次测试中观察到的最大延迟，而"中断-任务"一行展示了在处理程序通知高优先级任务时观察到的最大延迟。由于Tock不支持定义应用程序中断处理程序，我们改为将处理程序注册为其内核空间中的一个capsule。当我们通过两个任务作为系统后台工作负载相互上下文切换测量获得延迟。得益于软锁机制，Hopter在中断处理程序中响应时具有最低且最稳定的延迟。当用任务处理中断时，由于上下文切换较慢，Hopter变得比FreeRTOS慢。在这两种情况下，Hopter都比Tock快几个数量级。

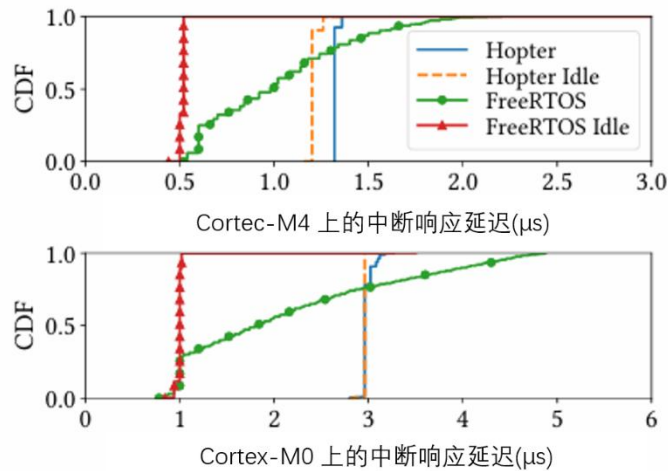


图 6-1 在 Cortex-M4 上使用软锁时，Hopter 的中断响应延迟几乎是恒定的。后台工作负载由于对指令和字面值缓存的压力导致延迟略有增加。由于在 M0 上实现原子操作（即禁用中断）的方式，Hopter 在 Cortex-M0 上的延迟有较小的方差。相比之下，FreeRTOS 的延迟对后台工作负载敏感，因为它在关键部分内屏蔽了中断。

Hopter的软锁在保持稳定的中断响应延迟方面是有效的，无论后台工作负载如何，如图6-1所示。负载下数值的略微增加是由于闪存中ART加速器[59]中缓存的指令和字面值的替换。相比之下，FreeRTOS的延迟对后台工作负载敏感。在其关键部分屏蔽中断会导致中断响应出现延迟，如果CPU以较低的频率（如在M0上）运行或操作系统负载较重，这种延迟会加剧。

Hopter在M0上运行时响应迅速，因为原子操作仅在短暂的常数时间内禁用中断。M0上的速度下降主要是由于CPU性能不如M4。为了测量由更长的函数序言和M0上的原子操作引起的开销，我们在M4板上运行相同的实验，但仅使用M0上可用的指令编译代码。Hopter的上下文切换和中断响应延迟增加了20%。当在M4上运行M0代码时，FreeRTOS比运行M4代码时稍快，因为M0代码在任务上下文切换期间不保存和恢复浮点寄存器。

6.2.3 致命错误恢复（Q4）

Hopter允许有故障的应用程序任务在资源回收期间执行任意的清理逻辑，这使得系统能够从更复杂的场景中恢复，并克服简单任务重启的限制。我们实现了一个概念验证应用程序，它由三个任务组成：一个通过串行接口从其他任务接收数据的串行服务器任务，以及两个定期向服务器任务发送数据的客户端任务。为了防止发送

任务之间的不良数据交错，客户端任务在发送数据前首先获取服务器的锁，并在完成后释放它。

我们故意在一个客户端任务持有锁时触发一个越界数组写入。在Hopter中，该故障表现为Rust panic，导致任务被重启。在任务展开期间，栈展开器调用锁守卫对象的析构函数来释放锁。重启后，两个客户端任务都正常进行。相比之下，在Tock上，如果写入地址落在任务允许的地址范围内，它将成为任务内部的静默数据损坏。否则，Tock检测到故障并重启任务，但锁不会被释放，随后导致死锁。由于FreeRTOS没有安全保护，越界写入要么导致系统范围内的数据损坏，要么触发导致系统挂起或重置的硬件故障。

6.3 无人机飞行控制



图6-2 Crazyflie 2.1，一款商业现成的微型无人机。我们实现了一个使用 Hopter 的飞行控制系统，以评估其支持实时应用的能力。

我们使用Hopter开发了一个飞行控制系统，以在真实应用中测量其资源使用和CPU负载的开销，并量化中断响应和任务恢复的延迟，展示其对实时工作负载的适用性。该程序在商业可用的Crazyflie 2.1硬件平台[6]上运行，如图6-2所示。无人机由STM32F405RG微控制器驱动，该微控制器具有最大时钟速度为168 MHz的Cortex-M4F CPU、192 KiB的SRAM和1 MiB的闪存存储。

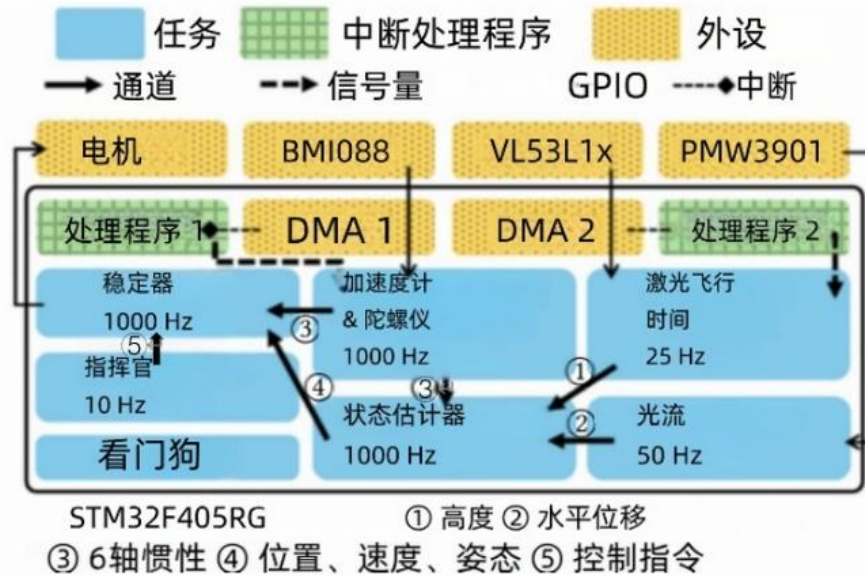


图 6-3 飞行控制系统由中断处理程序和高频率运行的周期性任务组成。任务和程序通过通道和信号量同步。外设通过 GPIO 引脚连接。

表 6-2 飞行控制系统的二进制大小、SRAM 使用和 CPU 负载。SRAM 使用不包括大小可配置的函数调用堆栈。CPU 负载是无人机悬停时 100 次测量的平均值。

	闪存 (KiB)	SRAM (KiB)	CPU
FreeRTOS	133.56	27.93	36.5%
Hopter	213.53	7.51	52.3%
Hopter-unwind	173.43	7.51	48.8%
Hopter-seg-stack	147.80	7.35	47.5%
Hopter-soft-lock	141.90	7.00	45.5%
Hopter-bare	136.95	6.52	48.4%

我们用 Rust 实现了飞行控制系统，大约包含 10,000 行代码。其中约 6,700 行是从开源库 [9, 51, 52] 手动翻译过来的外设驱动程序代码。我们使用 stm32-rs [30] 的 HAL 库来访问外设。飞行控制系统包含 22 条不安全的 Rust 语句，主要用于中断配置。

如图 6-3 所示，飞行控制系统包括七个周期性任务，它们共同管理飞行控制。三个任务分别负责从惯性传感器、高度传感器和光流传感器读取数据。其中两个通过直接内存访问（DMA）读取。随后，状态评估器任务通过 Hopter 提供的通道同步原语获取收集的数据，并使用卡尔曼滤波算法计算无人机的位置和姿态。最后，稳定器任务利用评估器的输出来调节四个螺旋桨电机的功率分配，旨在保持无人机

的稳定性并遵循来自指挥者任务的命令。为了操作安全，看门狗任务监控其他飞行控制任务，如果任何任务超过一秒钟无响应，看门狗将关闭所有电机。

6.3.1 系统大小和 CPU 负载 (Q1/Q2/Q3)

表6-2显示了飞行控制系统的闪存和SRAM使用情况，以及无人机悬停时的CPU负载。我们对Hopter的闪存开销进行了如下分解，其中带下划线的数字是常数，而其他数字随二进制文件大小增长。用于栈内存安全的函数序言和堆栈扩展运行时分别带来3.14 KiB和2.75 KiB的闪存开销。为了能够通过栈展开实现资源回收，栈展开器增加了9.59 KiB，着陆板增加了7.50 KiB，展开表增加了8.53 KiB。为了允许从栈溢出中恢复，析构函数插桩使代码大小增加了27.80 KiB，展开表大小增加了4.70 KiB。禁用基于noundwind函数属性的编译器优化使代码大小和展开表大小分别再增加了2.97 KiB和4.64 KiB。

我们还与一个具有类似程序结构的基于FreeRTOS的实现进行了比较。FreeRTOS的版本需要与Hopter的"bare"版本类似的闪存存储。其较高的SRAM使用是由于使用了较大的静态缓冲区，其较低的CPU负载是由于使用了高度优化的数学库和用于访问光流传感器的SPI总线的DMA驱动程序实现。

Hopter的析构函数插桩 (§ 4.1.1) 在闪存大小和CPU负载方面产生了最大的开销，这是违反直觉的，因为插桩的代码似乎很短并且只应用于析构函数。然而，我们在生成的代码中观察到一个级联效应，导致体积膨胀和CPU负载增加。(1) 如果组合类型中任何字段的类型实现了drop，则内部和外部类型的析构函数都会被插桩。(2) ARMv7作为一种RISC架构，在访问任务局部变量之前必须将其地址加载到寄存器中，导致指令序列更长。(3) 有更多的寄存器溢出，进一步增加了指令序列的长度。(4) 更大的函数体导致编译器不选择内联一些析构函数，导致更多的函数调用和返回。(5) 一个外联的函数又包含了额外的序言。

然而，尽管有开销，析构函数插桩对于从栈溢出的正确恢复是必不可少的。我们对七个飞行控制任务进行了静态栈深度分析，忽略通过函数指针或trait对象的间接函数调用（占有调用的0.1%）。五个任务在析构函数内部达到各自的`最大栈大小`。如果没有插桩，在析构函数内部启动栈展开将导致系统重置。

6.3.2 飞行中中断响应延迟 (Q3)

我们按照与 § 6.2.2 相同的实验设置，在设置无人机悬停时测量中断响应延迟。使用软锁，10,000 次测量中的最大中断响应延迟为 $1.04 \mu\text{s}$ ，标准差为 $0.02 \mu\text{s}$ 。然而，如果没有软锁，由于存在禁用中断响应的临界区，最大延迟上升到 $7.56 \mu\text{s}$ ，标准差为 $0.57 \mu\text{s}$ 。

及时的中断响应对于无人机上的无线电至关重要。无人机采用一个 nRF51822 芯片，通过一个通用异步收发器（UART）串行接口（波特率高达 2 Mbps）将接收到的无线电数据包转发到主微控制器。在无人机使用的 Syslink 协议[7]中，数据包的长度由前四个字节决定，因此只有在接收到这些字节后才能启动 DMA。没有 DMA，系统必须在 UART 接口接收到一个字节后立即响应中断。

表 6-3 稳定器任务在致命错误时的恢复延迟和栈展开器工作量。错误是故意使用 `panic!()` 或某些函数递归后的栈溢出导致的。任务上下文重用和并发重启优化都可以加快在 `panic` 和栈溢出时的恢复。无论栈深度如何，并发重启都能保持恒定的恢复延迟。

	Panic	OF-recur-4	OF-recur-10
未优化 (μs)	197	268	402
上下文重用 (μs)	134	207	335
并发重启 (μs)	189	192	190
#栈帧	6	7	13
#着陆板	2	6	12

考虑到每个数据字节携带一个起始位和一个停止位的开销，超过 $5 \mu\text{s}$ 的中断响应延迟将导致接收移位寄存器[60]超限，从而导致数据丢失。只有使用软锁，无线电模块才能正常工作。

6.3.3 飞行中致命错误容错 (Q4)

我们故意在无人机悬停时向飞行控制系统引入 `panic`，以评估 Hopter 从致命错误中恢复系统的能力。当我们将 `panic` 引入中断处理程序或除稳定器任务外的飞行控制任务时，无人机没有可见的扰动。当稳定器任务发生 `panic` 时，无人机会绕偏航轴旋转约 20 度并下降几厘米，但在任务重启后仍能悬停。稳定器对致命错误最敏感，因为它直接调节电机的功率。

我们还测量了稳定器在不同致命错误下的恢复延迟。对于 `panic` 测试，我们在更新稳定器状态的代码中放置一个 `panic!()` 语句，模拟数组越界访问或失败的断言。对

于栈溢出测试，我们插入一个对递归函数的调用，该函数定义一个具有析构函数的局部对象，该对象在4次或10次递归后将导致栈溢出。表6-3列出了每种测试用例的延迟，以及需要展开的栈帧数量和调用的着陆板数量。恢复延迟是从错误发生到重启任务实例的入口函数执行之间经过的时间。堆栈扩展会产生额外的20 μ s延迟。

测量结果表明，任务上下文重用和并发重启优化都能减少恢复延迟。当栈较浅时，任务上下文重用优于并发重启，但当栈较深时，并发重启通过保持恒定的延迟实现了更快的恢复。

在所有三种情况下，稳定器都能在下一次执行间隔之前恢复。观察到的无人机的扰动是由于新的稳定器任务实例没有任何命令可以执行。电机保持在零功率，直到接收到下一个命令。

6.4 IoT 网关

我们使用STM32F072B-Discovery板为物联网（IoT）设备开发了一个网关系统，以展示Hopter即使在无MPU的Cortex-M0微控制器上也能抵御运行时错误的健壮性。该网关由一个数据平面和一个控制平面组成，如图6-4所示。数据平面通过一个中断驱动的UART接口接收来自入站连接的数据包，并将它们转发到出站的UART连接。作为数据平面的一部分，网关任务验证接收到的数据包的完整性，更新运行时计数器，并可能在转发前根据规则修改数据包。控制平面通过另一个UART接口建立基于文本的控制会话。用户可以发送命令来查询已转发和损坏的数据包数量、统计特定字节模式的出现次数、重置计数器以及更改转发规则。这些规则允许用户指定转义字节，并根据数据包类型过滤数据包。作为控制平面的一部分，控制台任务解析和执行命令，并响应用户操作。

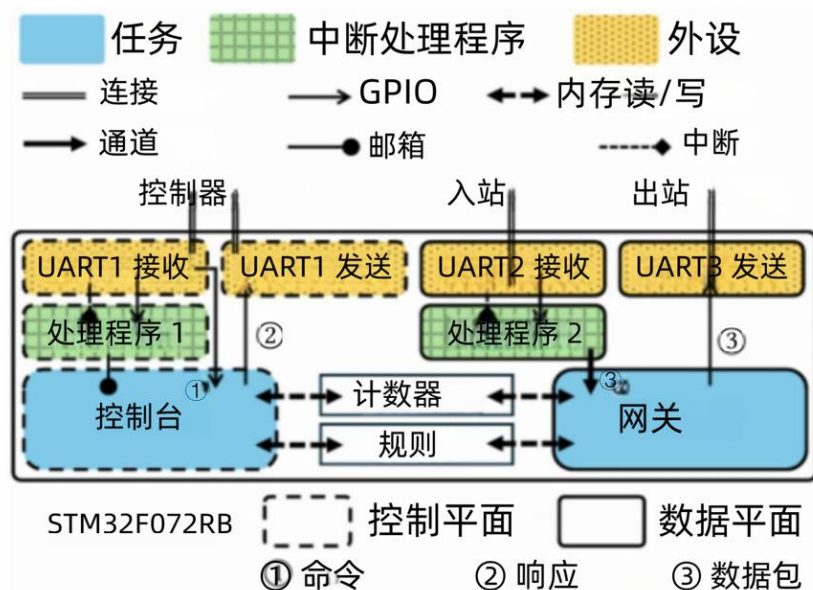


图 6-4 该网关由数据平面和控制平面组成。数据平面将数据包从入站连接转发到出站连接。得益于 Hopter 的健壮性特性, 控制平面中的错误不会影响数据平面的运行。

网关使用一个开源HAL库[29], 包含360行Rust代码, 其中只有8行不安全的代码用于配置外设。编译后的二进制文件需要60.0 KiB的闪存和大约1.3 KiB的SRAM (不包括堆栈)。相比之下, 基于FreeRTOS的实现需要21.0 KiB的闪存和大约3.0 KiB的SRAM。

Hopter将数据平面连接与控制平面致命错误隔离开来。为了触发错误，我们让控制台任务在接收命令时跳过检查缓冲区的剩余空闲大小。一个长命令会使缓冲区溢出，导致控制台任务panic并重启。同时，数据平面继续转发数据包，不受干扰。数据包转发延迟不受panic恢复过程的影响。相比之下，相同的缓冲区溢出错误会静默地损坏数据或导致基于FreeRTOS的系统崩溃。由于缺少MPU，Tock不支持此板。

第7章 相关工作

使用Rust实现内存安全：与Hopter使用Rust类似，其他操作系统也使用了Rust在内核和应用程序中实现内存安全。最近两项针对个人电脑和服务器的研究，Theseus[8]和RedLeaf[50]，使用Rust实现内存安全，但它们仍然依赖内存管理单元（MMU）放置保护页来防止栈溢出。嵌入式操作系统RIOT[5]、Embassy[11]和RTIC[12]，以及Tock[41]的内核空间，在微控制器上使用Rust实现内存安全。RIOT支持用Rust编写的线程化任务，但同样使用MPU/PMP设置保护区域来防止栈溢出。正如 § 4.1.2中所讨

论的，这种方法不允许从栈溢出中恢复。Embassy、RTIC和Tock的内核空间使用放置在SRAM区域下边界的单一向下增长的堆栈来防止栈溢出破坏内存。然而，它们只支持受限的调度模式，并且无法从栈溢出中恢复。相比之下，Hopter仅通过软件实现内存安全，支持线程化任务的任意调度模式，并且可以从栈溢出中恢复。

致命错误恢复：很少有适用于微控制器的嵌入式操作系统提供与Hopter从致命错误中恢复能力相关的恢复机制。Tock[41]可以自动重启因非法内存访问而失败的任务。Zephyr[27]也可以在此类错误时终止任务，但要求应用程序单独注册错误处理程序。两者都不支持自动清理以实现优雅的任务终止。相比之下，Hopter运行应用程序清理逻辑（析构函数），使系统能够避免后续错误，如死锁。

零延迟中断处理：Hopter使用软锁进行零延迟中断处理，与之前的嵌入式操作系统如eCos[47]、PEASE[58]、PURE[57]和SMX[48]不同，后者将中断处理分为上半部和下半部以避免操作系统中断屏蔽。这种分割设计需要用于延迟操作（下半部）的全局序列化队列，这些操作引用了操作系统对象，因此需要不安全的Rust，因为编译器无法验证引用的生命周期。此外，Hopter的软锁可以通过仅在发生争用时（很少见）推迟操作来提高性能，从而主要通过快速路径减少CPU负载。

结 论

本文描述了Hopter，一个为微控制器设计的基于Rust的嵌入式操作系统。Hopter为应用程序提供内存安全、系统健壮性和高中断响应能力，同时仅需应用最小程度的适配。Hopter通过在使用编译时代码插桩和运行时操作系统支持的FS-语义下执行Rust代码，来防止栈溢出并将此类错误转换为Rust panic。通过将所有致命错误统一为panic，Hopter执行栈展开以从失败的应用程序任务或中断处理程序中回收资源，并且可以自动重启失败的任务。为了确保对中断的及时响应，Hopter引入了一种称为软锁的新颖机制来实现零延迟中断处理。Hopter非常适合资源受限的微控制器，并支持实时工作负载的错误恢复。从栈溢出中恢复的机制也适用于在服务器中运行的进程，软锁在降低响应进程信号的延迟方面也可能有用。

参考文献

参考文献书写规范

- [1] Amazon.com, Inc. 2020. FreeRTOS: Real-time operating system for microcontrollers and small microprocessors. <https://www.freertos.org>. Version 10.3.1. Accessed: 2025-04-24.
- [2] Arm Limited. 2021. ARMv7-M Architecture Reference Manual (Issue E.e). [https://developer.arm.com/documentation/ddi0403/ee/Chapter B3.5: Protected Memory System Architecture, PMSAv7](https://developer.arm.com/documentation/ddi0403/ee/Chapter%20B3.5%3A%20Protected%20Memory%20System%20Architecture,PMSAv7). Accessed: 2025-04-24.
- [3] Arm Ltd. 2023. Arm Compiler Version 6.6.5 armclang Reference Guide. [https://developer.arm.com/documentation/dui0774/1/Section 2.28](https://developer.arm.com/documentation/dui0774/1/Section%202.28). Accessed: 2025-04-24.
- [4] AWS open source. 2024. FreeRTOS Documentation. [https://www.freertos.org/Documentation/00-Overview Chapter: FreeRTOS Stack Usage and Stack Overflow Checking](https://www.freertos.org/Documentation/00-Overview%20Chapter%3A%20FreeRTOS%20Stack%20Usage%20and%20Stack%20Overflow%20Checking). Accessed: 2025-04-24.
- [5] Emmanuel Baccelli, Cenk Gündoğan, Oliver Hahm, Peter Kietzmann, Martine Slenders, Hauke Petersen, Kaspar Schleiser, Thomas C Schmidt, and Matthias Wählisch. 2018. RIOT: An open source operating system for low-end embedded devices in the IoT. IEEE Internet of Things Journal (2018).
- [6] Bitcraze. 2020. Crazyflie: A flying open development platform. <https://www.bitcraze.io/>.
- [7] Bitcraze. 2024. Syslink protocol version 2024.10. <https://www.bitcraze.io/documentation/repository/crazyflie2-nrf-firmware/2024.10/protocols/syslink/> Accessed: 2025-04-24.
- [8] Kevin Boos, Namitha Liyanage, Ramla Ijaz, and Lin Zhong. 2020. Theseus: an experiment in operating system structure and state management. In Proc. USENIX OSDI.
- [9] Bosch. 2024. BMI088. <https://github.com/BoschSensortec/BMI08x-Sensor-API>. Version 1.9.0. Accessed: 2025-04-24.
- [10] Ching-Han Chen, Ming-Yi Lin, and Chung-Chi Liu. 2018. Edge computing gateway of the industrial internet of things using multiple collaborative microcontrollers. IEEE Network (2018).
- [11] Embassy Project Contributors. 2023. Embassy: The next-generation framework for embedded applications. <https://embassy.dev>. Accessed: 2025-04-24.
- [12] RTIC Contributors. 2023. RTIC: The hardware accelerated Rust RTOS. <https://rtic.rs/2/book/en/>. Accessed: 2025-04-24.
- [13] Jonathan Corbet. 2022. A first look at Rust in the 6.1 kernel. <https://lwn.net/Articles/910762/>. Linux Weekly News (10 2022).
- [14] Microsoft Corporation. [n.d.]. Windows Drivers in Rust. <https://github.com/microsoft/windows-drivers-rs>. Accessed: 2025-04-24.
- [15] MITRE Corporation. 2019. CVE-2019-25001: Flaw in CBOR deserializer allows stack overflow. <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2019-25001>. Accessed: 2025-04-24.
- [16] MITRE Corporation. 2020. CVE-2020-35858: Parsing a specially crafted message can result in a stack overflow. <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2020-35858>. Accessed: 2025-04-24.
- [17] MITRE Corporation. 2021. CVE-2021-31572: FreeRTOS before 10.4.3 has an integer overflow for a stream buffer. <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-31572>. Accessed: 2025-04-24.

- [18] MITRE Corporation. 2021. CVE-2021-32020: FreeRTOS before 10.4.3 has insufficient bounds checking during management of heap memory. <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-32020>. Accessed: 2025-04-24.
- [19] MITRE Corporation. 2024. CVE-2024-36760: A stack overflow vulnerability found in version 1.18.0 of rhai. <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-36760>. Accessed: 2025-04-24.
- [20] Alex Crichton, Jeehoon Kang, Aaron Turon, and Taiki Endo. 2024. Tools for concurrent programming. <https://crates.io/crates/crossbeam>. Version 0.8.4. Accessed: 2025-04-24.
- [21] Amanieu d'Antras. 2024. Intrusive collections for Rust (linked list and red-black tree). <https://crates.io/crates/intrusive-collections>. Version 0.9.7. Accessed: 2025-04-24.
- [22] The Rust Security Advisory Database. 2018. RUSTSEC-2018-0005: Uncontrolled recursion leads to abort in deserialization. <https://rustsec.org/advisories/RUSTSEC-2018-0005.html>. Accessed: 2025-04-24.
- [23] The Rust Security Advisory Database. 2023. RUSTSEC-2023-0080: Buffer overflow due to integer overflow in transpose. <https://rustsec.org/advisories/RUSTSEC-2023-0080.html>. Accessed: 2025-04-24.
- [24] The Rust Security Advisory Database. 2024. RUSTSEC-2024-0012: Stack overflow during recursive JSON parsing. <https://rustsec.org/advisories/RUSTSEC-2024-0012.html>. Accessed: 2025-04-24.
- [25] Hopter Developers. 2024. Hopter Embedded OS. <https://github.com/hopter-project/hopter>. Accessed: 2025-04-24.
- [26] Redox Developers. 2015. Redox- Your Next(gen) OS. <https://www.redox-os.org/>. Accessed: 2025-04-24.
- [27] Zephyr Developers. 2016. Zephyr Project. <https://github.com/zephyrproject-rtos/zephyr>. Accessed: 2025-04-24.
- [28] Guo Dong, Wang Hongpei, Gao Song, and Wang Jing. 2011. Study on adaptive front-lighting system of automobile based on microcontroller. In Proc. IEEE Int.Conf. Transportation, Mechanical, and Electrical Engineering (TMEE).
- [29] Daniel Egger. 2021. Peripheral access API for STM32F0 series microcontrollers. <https://crates.io/crates/stm32f0xx-hal>. Version 0.18.0. Accessed: 2025-04-24.
- [30] Daniel Egger. 2024. Peripheral access API for STM32F4 series microcontrollers. <https://crates.io/crates/stm32f4xx-hal>. Version 0.22.1. Accessed: 2025-04-24.
- [31] Bill Fleming. 2011. Microcontroller units in automobiles [automotive electronics]. IEEE Vehicular Technology Magazine (2011).
- [32] Free Software Foundation, Inc. 2023. GCC 15.0.0 Manual. [https://gcc.gnu.org/onlinedocs/gcc/Section 3.12, Program Instrumentation Options](https://gcc.gnu.org/onlinedocs/gcc/Section%203.12,%20Program%20Instrumentation%20Options). Accessed: 2025-04-24.
- [33] Adam Greig. 2023. Low level access to Cortex-M processors. <https://crates.io/crates/cortex-m>. Version 0.7.7. Accessed: 2025-04-24.
- [34] Mehedi Hasan, Maruf Hossain Anik, and Sharnali Islam. 2018. Microcontroller based smart home system with enhanced appliance switching capacity. In Proc. IEEE HCT Information Technology Trends (ITT).

- [35] Robert Hieb, R Kent Dybvig, and Carl Bruggeman. 1990. Representing control in the presence of first-class continuations. ACM SIGPLAN Notices (1990).
- [36] Free Software Foundation Inc. 2023. Exception handling and frame unwind runtime interface routines.
<https://github.com/gcc-mirror/gcc/blob/721cdcd1ddde0738deb08895e113a8db84187a14/libgcc/unwind.inc>. Accessed: 2025-04-24.
- [37] FortuneBusiness Insights. 2024. Embedded Systems Market Size, Share & COVID-19 Impact Analysis, By Component, By Type, By Application, and Regional Forecast, 2023-2030.
<https://www.fortunebusinessinsights.com/embedded-systems-market-108767>. Accessed: 2025-04-24.
- [38] Future Market Insights. 2024. Embedded System Market Analysis from 2024 to 2034.
<https://www.futuremarketinsights.com/reports/embedded-system-market>. Accessed: 2025-04-24.
- [39] Global Market Insights. 2024. Embedded Systems Market Size- By Component, By Application, By Function & Forecast, 2024- 2032.
<https://www.gminsights.com/industry-analysis/embedded-system-market>. Accessed: 2025-04-24.
- [40] Prateek Khurana, Rajat Arora, and Manoj Kr Khurana. 2014. Microcontroller based implementation of Electronic Stability Control for automobiles. In Proc.IEEE Int. Conf. Advances in Engineering & Technology Research.
- [41] Amit Levy, Bradford Campbell, Branden Ghena, Daniel B Giffin, Pat Pannuto, Prabal Dutta, and Philip Levis. 2017. Multiprogramming a 64kb computer safely and efficiently. In Proc. ACM SOSP.
- [42] LLVM. 2025. AttributorAttributes.cpp- Attributes for Attributor deduction.
https://llvm.org/doxygen/AttributorAttributes_8cpp_source.html. Accessed: 2025-04-24.
- [43] LLVM. 2025. FunctionAttrs.cpp- Pass which marks functions attributes.
https://llvm.org/doxygen/FunctionAttrs_8cpp_source.html. Accessed: 2025-04-24.
- [44] Zhiyao Ma, Guojun Chen, and Lin Zhong. 2023. Panic Recovery in Rust-based Embedded Systems. In Proc. ACM PLOS.
- [45] Zhiyao Ma and Lin Zhong. 2023. Bringing Segmented Stacks to Embedded Systems. In Proc. ACM HotMobile.
- [46] Aleksander Malinowski and Hao Yu. 2011. Comparison of embedded system design for industrial applications. IEEE Trans. Industrial Informatics (2011).
- [47] Anthony J Massa. 2002. Embedded software development with eCos. Prentice Hall Professional.
- [48] Ralph Moore. 2005. Link Service Routines. Micro Digital (2005).
- [49] Mohamed Abd El-Latif Mowad, Ahmed Fathy, Ahmed Hafez, et al. 2014. Smart home automated control system using android application and microcontroller. Int. Journal of Scientific & Engineering Research (2014).
- [50] Vikram Narayanan, Tianjiao Huang, David Detweiler, Dan Appel, Zhaofeng Li, Gerd Zellweger, and Anton Burtsev. 2020. RedLeaf: Isolation and Communication in a Safe Operating System. In Proc. USENIX OSDI.
- [51] Pimoroni. 2024. PMW3901. <https://github.com/pimoroni/pmw3901-python/tree/main>. Version 1.0.0. Accessed: 2025-04-24.
- [52] Pololu. 2022. V5311x. <https://github.com/pololu/v5311x-arduino>. Version 1.3.1. Accessed: 2025-04-24.

- [53] LLVM Project. 2023. Handling Level 1. Implementation of C++ ABI Exception <https://github.com/llvm/llvm-project/blob/f42482def236999b0f7896c09cd714b708861c8b/libunwind/src/UnwindLevel1.c>. Accessed: 2025-04-24.
- [54] RISC-V. 2024. The RISC-V Instruction Set Manual: Volume II (Version 20240411). <https://riscv.org/technical/specifications/> Chapter 3.7: Physical Memory Protection. Accessed: 2025-04-24.
- [55] Rust on Embedded Devices Working Group et al. [n.d.]. The Embedded Rust Book. <https://docs.rust-embedded.org/book/>. Accessed: 2025-04-24.
- [56] Alice Ryhl. 2023. Rewriting Binder Driver in Rust. <https://lore.kernel.org/rust-for-linux/20231101-rust-binder-v1-0-08ba9197f637@google.com/>. Accessed: 2025-04-24.
- [57] Friedrich Schon, Wolfgang Schroder-Preikschat, Olaf Spinczyk, and Ute Spinczyk. 2000. On interrupt-transparent synchronization in an embedded object-oriented operating system. In Proc. IEEE Int. Symp. Object-Oriented Real-Time Distributed Computing (ISORC).
- [58] Wolfgang Schröder-Preikschat. 1994. The logical design of parallel operating systems. Prentice-Hall, Inc.
- [59] STMicroelectronics. 2020. RM0402 (Rev.6) Reference manual for STM32F412 advanced Arm-based 32-bit MCUs. Chapter 3.4.2: Adaptive real-time memory accelerator (ART Accelerator).
- [60] STMicroelectronics. 2024. RM0090 (Rev.21) Reference manual for STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 advanced Arm-based 32-bit MCUs. https://www.st.com/resource/en/reference_manual/rm0090-stm32f405415-stm32f407417-stm32f427437-and-stm32f429439-advanced-armbased-32bit-mcus-stmicroelectronics.pdf Chapter 30.3.3: Receiver-OverrunError. Accessed: 2025-04-24.
- [61] STMicroelectronics. 2024. STM32CubeF0 HAL driver MCU component. <https://github.com/STMicroelectronics/STM32CubeF0/releases/tag/v1.11.5>. Version 1.11.5. Accessed: 2025-04-24.
- [62] STMicroelectronics. 2024. STM32CubeF4 HAL driver MCU component. https://github.com/STMicroelectronics/stm32f4xx_hal_driver/releases/tag/v1.8.3. Version 1.8.3. Accessed: 2025-04-24.
- [63] Rust Tools Team. 2021. svd2rust. <https://github.com/rust-embedded/svd2rust>
- [64] Theseus Developers. 2023. Support for unwinding the call stack and cleaning up stack frames. <https://github.com/theseus-os/Theseus/blob/562a39cf6c662738f7718473f6bbc010970dce53/kernel/unwind/src/lib.rs>. Accessed: 2025-04-24.
- [65] Vishakha D Vaidya and Pinki Vishwakarma. 2018. A comparative analysis on smart home system to control, monitor and secure home, based on technologies like gsm, iot, bluetooth and pic microcontroller with zigbee modulation. In Proc.IEEE Int. Conf. Smart City and Emerging Technology (ICSCET).
- [66] Mathijs van de Nes and Joshua Barretto. 2023. Spin-based synchronization primitives. <https://crates.io/crates/spin>. Version 0.9.8. Accessed: 2025-04-24.
- [67] Rob Von Behren, Jeremy Condit, Feng Zhou, George C Necula, and Eric Brewer. 2003. Capriccio: scalable threads for internet services. ACM SIGOPS Operating Systems Review (2003)

附 录